AI INNOVATION CHALLENGE

Build a product feature using harness engineering and AI/ML models

A 48-hour team challenge for aspiring product engineers

The objective

Design and implement a product feature that uses AI/ML capabilities responsibly within a governed engineering harness. Teams should demonstrate how AI can improve a real user workflow while ensuring reliability, traceability, security, and human oversight.

The challenge evaluates both the feature experience and the surrounding system that makes AI/ML output safe, testable, observable, and production-ready.

Team	3-5 members (actual size may vary based on shortlisted candidates)
Duration	48 hours
Build	Working prototype, source code, documentation, and presentation recording
Demo	10-minute recording

What great solutions demonstrate

Product value comes from combining LLM reasoning with trusted context, deterministic controls, product workflows, validation, and human accountability - not from using an LLM alone.

Choose a product use case

Teams may select one of the following use cases. A solution should address a clear user problem and demonstrate an end-to-end workflow.

1	**Policy or Knowledge Assistant**	Answer employee or administrator questions using approved knowledge sources, provide citations, identify uncertainty, and escalate unsupported questions.
2	**Product Requirements-to-Test Assistant**	Convert a structured product requirement into acceptance criteria, test scenarios, edge cases, and traceable automated-test recommendations.
3	**Employee Onboarding**	Create a new-hire onboarding orchestrator that coordinates documents, equipment, payroll setup, benefits enrolment, training, and manager tasks based on worker profile and location.
4	**Recruiting**	Create a recruiting operations agent that screens for minimum job criteria, summarizes candidate evidence, schedules interviews, and prepares structured interview packs.
5	**Time & Attendance**	Create a timecard exception resolution agent that detects missed punches, overtime risks, schedule conflicts, and policy breaches, then proposes compliant corrections.

Suggested technical stack

Teams may use the options below or propose an equivalent stack that meets the challenge expectations.

Layer	Recommended options
Local development	Eclipse, IntelliJ, Visual Studio, or another integrated development environment
AI-assisted coding	Cursor, GitHub Copilot, or another AI coding assistant
Frontend	React / TypeScript
Backend	Node.js / TypeScript, Python FastAPI, or .NET
LLM integration	Open-source or open-weight LLMs. Verify the selected model's licence and acceptable-use terms before use.
Orchestration	Lightweight service orchestration; LangChain, LangGraph, or Semantic Kernel only where it adds clear value
Knowledge retrieval	Vector store or structured search over approved documents or mock data
Data storage	PostgreSQL, SQLite, MongoDB, or Elasticsearch
API contract	OpenAPI specification
Validation	JSON Schema, Zod, Pydantic, or equivalent typed validation
Testing	Unit, integration, API-contract, and evaluation tests
Delivery	GitHub repository with CI checks and README

Required deliverables

Submit a complete solution that makes the user value, engineering decisions, and evidence of quality easy to assess.

1	**GitHub repository**	Include clear setup and run instructions, plus a README describing the product problem, user journey, architecture, and key technical decisions.
2	**Demo recording**	Provide a 10-minute demonstration of the working prototype and an end-to-end user workflow. Add the demo video to the GitHub repository README.
3	**Five-slide final presentation**	Cover: (1) problem and user value, (2) product workflow and demo, (3) architecture and harness design, (4) safety, testing, and evaluation evidence, and (5) future improvements and expected business impact.

How submissions will be evaluated

Category	Weight	What judges will assess
Product value	25%	Clear user problem, practical workflow, and measurable outcome
Harness engineering	25%	Structured contracts, validation, guardrails, fallback design, and human oversight
LLM implementation	20%	Appropriate grounding, prompt design, retrieval quality, and useful model behavior
Engineering quality	15%	Code quality, API design, testing, observability, and documentation
Demo recording	15%	Clear demonstration, technical explanation, and evidence-based results

Build thoughtfully: ground AI in trusted context, validate its output, and keep people accountable for important decisions.